Ternary CNN Inference Engine

Architecture, Data Flow & Design Decisions — ResNet-18 on CIFAR-10 — Intel i5-13400F (AVX2+VNNI)

1. Complete Data Flow (Single Inference)

INPUT
FP32 Tensor [1, 3, 32, 32] = 12 KB
CIFAR-10 normalized: range [-2.18, +2.70]

↓

Conv1 (FP32) — 3→64ch, 3×3, s1p1
FP32 im2col [27 × 1024] = 108 KB
AVX2 FMA fmadd_ps, 8 floats/cycle
BN fused: scale folded into weights, bias adjusted

→

FP32 [1, 64, 32, 32] = 256 KB
range [-0.85, 1.15], mean 0.02

↓ ReLU (in-place AVX2 maxps)

Quantize FP32 → UINT8
AVX2 cvtps → packus (8 vals/iter)
scale = 0.009273 (calibrated offline)
u8 = round(fp32 / scale), clamp [0, 255]

→

UINT8 [1, 64, 32, 32] = **64 KB**
range [0, 124], 59% zeros, scale=0.0093
All activations stay uint8 from here

↓ Enters INT8-resident pipeline (8 ResNet blocks)

Per-Block Pipeline (repeated 8 times)

Block Input x
UINT8 [1, C, H, W]
+ scale (float32)

→

STEP 1: im2col (uint8 byte copies)
3×3 s1p1 specialized: memcpy per row (no per-element branches)
Output: col_u8 UINT8 [col_rows × spatial]
Layer1: [576 × 1024] = 576 KB

↓

STEP 2: SSE Repack (spatial-packed layout for dpbusd)
4 loads + 3 shuffles + 2 stores per 32-byte block
`unpacklo_epi8 → unpacklo_epi16` = 4×8 byte transpose
Output: col_packed UINT8 [jg × kg × 32B]
Layer1: [128 × 144 × 32] = 576 KB

→

Memory Layout
`packed[jg][kg][32B]`
32B = 8 spatial × 4 reduction
`[s0r0 s0r1 s0r2 s0r3`
`s1r0 s1r1 s1r2 s1r3`
`... s7r0 s7r1 s7r2 s7r3]`

↓

STEP 3: dpbusd GEMM (16-OC tiling)
AVX-VNNI `_mm256_dpbusd_epi32(acc, act_u8, wt_s8_broadcast)`
32 MACs per instruction, 1-cycle throughput
Weights INT8 {-1,0,+1} packed as int32 (4 per word, broadcast)
Accumulator: INT32 × 16 OCs in YMM registers

→

INT32 accumulators
16 OCs × 8 spatial
= 16 YMM registers
(all 16 used — at register limit)

↓

STEP 4: Fused Epilogue (inside GEMM output loop, zero extra passes)
Mode 1 (conv1): i32 → f32 (dequant × scale + bias) → ReLU → u8
Mode 2 (conv2): i32 → f32 → + *residual* → ReLU → u8
Dequant: `real = int32_acc × act_scale × oc_scale[oc] + oc_bias[oc]`
Requant: `u8 = clamp(round(real / out_scale), 0, 255)`

→

Block Output
UINT8 [1, OC, OH, OW]
+ new scale (calibrated)
Becomes next block's input

↓ After 8 blocks: [1, 512, 4, 4] uint8

Global AvgPool
Dequant u8→int32 sum × scale / spatial
Output: FP32 [512]

→

FC Layer
FP32 matmul [512 → 10]
AVX2 FMA

→

Logits [10]
argmax → prediction

2. Activation Sizes Through the Network

Stage	Shape	Dtype	Size	Scale	Value Range	% Zeros
Input	[1, 3, 32, 32]	float32	12 KB	—	[-2.18, 2.70]	—
After conv1+ReLU	[1, 64, 32, 32]	float32	256 KB	—	[0, 1.15]	45%
Quantized	[1, 64, 32, 32]	**uint8**	**64 KB**	0.0093	[0, 124]	59%
layer1.0 out	[1, 64, 32, 32]	uint8	64 KB	0.0122	[0, 132]	30%
layer1.1 out	[1, 64, 32, 32]	uint8	64 KB	0.0123	[0, 114]	32%
layer2.0 out (stride 2)	[1, 128, 16, 16]	uint8	32 KB	0.0117	[0, 82]	56%
layer2.1 out	[1, 128, 16, 16]	uint8	32 KB	0.0134	[0, 105]	39%
layer3.0 out (stride 2)	[1, 256, 8, 8]	uint8	16 KB	0.0069	[0, 84]	72%
layer3.1 out	[1, 256, 8, 8]	uint8	16 KB	0.0071	[0, 98]	43%
layer4.0 out (stride 2)	[1, 512, 4, 4]	uint8	8 KB	0.0160	[0, 82]	70%
layer4.1 out	[1, 512, 4, 4]	uint8	8 KB	0.0384	[0, 133]	76%
Pooled	[512]	float32	2 KB	—	[0, 3.14]	—

Key observation: Activations are 4× smaller as uint8 vs float32. Sparsity increases in deeper layers (76% zeros in layer4.1) but we don't exploit it — dpbusd processes zeros at same speed.

3. Weight Storage & Packing

On Disk: 2-bit I2_S Packed (3.56 MB total)

```
Per byte: 4 ternary weights, MSB first
  +1 → 0b01
   0 → 0b00
  -1 → 0b11
Per OC: alpha (float32) = mean(|w| where |w| > threshold)
```

12.5× compression vs FP32 (44.69 MB)

In Memory: dpbusd-ready int32 packed

```
At model load: unpack 2-bit → int8 {-1,0,+1}
Then: pack4_i8(w[g*4+0..3]) → one int32
  low byte  = w[g*4+0] as uint8
  next byte = w[g*4+1] as uint8
  ...
Broadcast via _mm256_set1_epi32(packed_word)
Layout: weights_packed[OC][num_k_groups]
```

Layer Group	OC×IC	Kernel	col_rows	Padded/Groups	Weight Size (packed)	+1 / 0 / -1 dist
layer1 (4 convs)	64×64	3×3	576	576 / 144	36 KB/conv	~20% / 50% / 30%
layer2 (4 convs)	128×64..128	3×3	576..1152	576..1152 / 144..288	72..144 KB	~25% / 45% / 30%
layer3 (4 convs)	256×128..256	3×3	1152..2304	1152..2304 / 288..576	288..576 KB	~25% / 44% / 31%
layer4 (4 convs)	512×256..512	3×3	2304..4608	2304..4608 / 576..1152	1152..2304 KB	~24% / 50% / 26%
Shortcuts (3)	128..512 × 64..256	1×1	64..256	64..256 / 16..64	8..128 KB	FP32→INT8 quantized

4. The dpbusd Spatial-Packed Layout (Key Innovation)

Before: Reduction-Major (Exp 2, 17ms)

```
For each (OC, spatial_pos):
  acc = dpbusd(act[0:31], wt[0:31])  // 32 MACs
  result = horizontal_reduce(acc)     // EXPENSIVE
  // shuffle+hadd: saturated port 6 at 55%
```

Problem: dpbusd produces 8 partial sums in lanes. Need horizontal reduction to get one scalar. Shuffle instructions bottleneck port 6.

After: Spatial-Packed (Exp 7, 9ms → final)

```
For each OC_group(16), spatial_group(8):
  acc[16] = {0}   // 16 OC accumulators
  for kg in reduction_groups:
    a = load 32B  // 8 spatial × 4 reduction
    for oc in 0..15:
      w = broadcast weight_packed[oc][kg]
      acc[oc] = dpbusd(acc[oc], a, w)
  // NO horizontal reduction needed!
  // Each lane = one spatial position's result
```

Each YMM lane accumulates a *different* spatial position. Results land directly in the correct lane. CPI: 0.294 → 0.179.

dpbusd Instruction Detail

```
_mm256_dpbusd_epi32(acc, activation, weight)
  // For each of 8 groups of 4 bytes:
  //   acc[i] += u8[i*4+0]*s8[0] + u8[i*4+1]*s8[1] + u8[i*4+2]*s8[2] + u8[i*4+3]*s8[3]
  // activation: 32 bytes = 8 lanes × 4 reduction values (uint8, from col_packed)
  // weight: 4 bytes broadcast to all lanes (int8, from weights_packed)
  // acc: 8 × int32 partial sums (one per spatial position)
  // Throughput: 1 instruction/cycle on Raptor Lake = 32 MACs/cycle
```

5. Scale Arithmetic (Static Quantization)

The Scale Chain

```
Real value reconstruction:
  real_activation = uint8_value × act_scale          (per-tensor, from calibration)
  real_weight     = int8_weight × alpha[oc]           (per-channel, from TWN training)
  real_conv_out   = SUM(real_act × real_wt)           = SUM(u8 × i8) × act_scale × alpha[oc]
                  = int32_acc × act_scale × alpha[oc]

With fused BN:
  bn_out = real_conv_out × bn_scale[oc] + bn_bias[oc]
         = int32_acc × (act_scale × alpha[oc] × bn_scale[oc]) + bn_bias[oc]
         = int32_acc × (act_scale × oc_scale[oc]) + oc_bias[oc]

  where: oc_scale[oc] = alpha[oc] × bn_scale[oc]    ← precomputed at model load
         oc_bias[oc]  = bn_bias[oc]                   ← precomputed at model load

Requantize to uint8:
  u8_out = clamp(round(relu(bn_out) / out_act_scale), 0, 255)

  where: out_act_scale = calibrated scale for this layer's output activation range
```

What's precomputed (at model load)

- `oc_scale[oc]` = alpha × bn_scale (per OC)
- `oc_bias[oc]` = bn_bias (per OC)
- `weights_packed[oc][kg]` = 4 int8 per int32
- Shortcut weights: FP32 → INT8 symmetric quantization

What happens at runtime (per inference)

- `dequant = act_scale × oc_scale[oc]` (1 multiply per OC)
- `result = int32_acc × dequant + bias` (FMA per element)
- ReLU: `max(result, 0)`
- Requant: `round(result / out_scale)`, clamp [0,255]

6. Crucial Design Decisions

Decision 1: Treat ternary as generic INT8 (not special-cased)

Alternatives tried: INT16 add/sub (17.5ms), sign_epi16 (17.5ms), sparse pos/neg lists (17.9ms), FP32 add/sub (81ms), 4-bit sign+add (32.9ms)
Why INT8 wins: dpbusd does 32 MACs/instruction with 1-cycle throughput regardless of weight values. The hardware multiplier runs at the same speed for {-1,0,+1} as for any int8. "Skipping" the multiply with add/sub actually *reduces* throughput (8 ops/instr for add_epi32 vs 32 for dpbusd). The multiplier is free.
Impact: 7× faster than the best ternary-specific approach. This single decision defines the entire engine architecture.

Decision 2: Spatial-packed layout (8 spatial × 4 reduction per dpbusd)

Alternative: Reduction-major layout (32 reduction values per dpbusd, horizontal reduce after).
Why spatial-packed wins: Horizontal reduction (shuffle+hadd) saturated port 6 at 55%. By placing different spatial positions in different lanes, results land directly in the correct lane. Zero horizontal reductions needed.
Impact: CPI 0.294 → 0.179. Retiring 54% → 82%. The single biggest microarchitectural win (1.75× GEMM speedup).

Decision 3: INT8-resident execution (activations stay uint8 between ALL layers)

Alternative: Dynamic quantization per layer (FP32 between layers, quantize on entry to each conv).
Why static INT8 wins: Dynamic quantization requires: (a) finding max value (reduction over entire tensor), (b) dividing by max, (c) converting to uint8. For layer1 (64×32×32), that's ~200K FP32 operations just for quantization. Static calibration moves this cost offline.
Impact: Eliminated 24.5% FP32 im2col overhead + 9.7% findmax overhead. im2col operates on bytes (4× less data).

Decision 4: Two-pass im2col + repack (NOT fused)

Alternatives tried: (a) Fused implicit GEMM — gather bytes inside GEMM loop (Exp 14: 58ms). (b) Fused im2col→col_packed — skip col_u8 (4.71ms vs 4.48ms). (c) Unrolled 9-position im2col with inline AVX2 copies (4.58ms vs 4.48ms).
Why two-pass wins: Sequential bulk memory operations (memcpy for im2col, SSE shuffle for repack) are extremely efficient on modern CPUs. Fused approaches add per-element address computation that destroys the sequential access pattern. The icache/DSB also penalizes code bloat from inlining.
Impact: 21% of runtime. This is the gap to ONNX INT8 Static (which uses JIT-generated fused implicit GEMM with prefetch hints).

Decision 5: Fused epilogues (residual add + ReLU + requant in GEMM output loop)

Alternative: Separate passes for dequant, residual add, ReLU, and requant.
Why fused wins: The int32 accumulator is already in a YMM register. Dequant (cvtepi32+fmadd), residual add, ReLU (maxps), and requant (cvtps+packus) all happen while data is in registers. Zero intermediate tensors written to memory.
Impact: Eliminated 3 separate passes over the output tensor. Especially important for mode 2 (conv2 + residual) where unfused would require a temporary FP32 tensor.

Decision 6: 16-OC tiling (not 8, not 32)

Why 16: Raptor Lake has 16 YMM (256-bit) registers. With 16 OC accumulators, each uses 1 register. The activation load (1 register) is reused across all 16 OCs. Going to 32 OC would require spilling to stack; 8 OC wastes half the register file.
Impact: Each 32B activation load is amortized across 16 OC channels. Activation bandwidth reduced 16× compared to no tiling.

Decision 7: Shortcut convs via dpbusd (not FP32 GEMM)

Alternative: Keep shortcut 1×1 convs as FP32 (they have FP32 weights from training).
Why INT8 wins: Quantize FP32 weights to INT8 at model load (per-channel symmetric). Route through the same spatial-packed dpbusd path. This eliminated all FP32 GEMM except the first conv.
Impact: Engine::forward overhead dropped from 20.8% → 3.0%. dpbusd processes 4× more values per instruction than FP32 FMA.

Decision 8: Preallocated buffers + DefaultInitAlloc (zero malloc on hot path)

Problem: Original code created new Tensor/TensorU8 (via std::vector) in every conv call. That's ~19 malloc+free per inference + zero-initialization of every output buffer.
Fix: (a) Preallocate 3 uint8 + 2 float32 activation buffers at max size. Ping-pong between them. `reshape()` reuses memory if capacity suffices. (b) `DefaultInitAlloc` skips zero-fill since GEMM epilogue writes every element.
Impact: malloc disappeared from perf profile. memset dropped from 1.4% to <0.3%.

7. VTune Profile (Final Engine, 1-Thread)

Function	Time %	CPI	Retiring %	FE Bound %	BE Bound %	Mem Bound %	Key Bottleneck
dpbusd GEMM (constprop.0)	43.4%	0.179	82.2%	1.3%	16.7%	5.8%	Core bound 11% (register pressure)
dpbusd GEMM (constprop.2)	34.6%	0.178	84.6%	2.3%	13.0%	6.9%	Core bound 6%
im2col_u8	10.4%	0.230	62.1%	**36.3%**	1.7%	0.4%	Front-end bound (DSB misses)
memcpy (im2col rows)	5.0%	0.312	39.2%	**40.5%**	20.6%	27.6%	FE bound + store bound
Engine::forward (conv1 FP32)	4.2%	0.366	44.3%	8.3%	43.2%	50.9%	Memory bound (FP32 data)
GEMM shortcut (constprop.1)	1.3%	0.186	83.9%	4.8%	18.4%	9.2%	Small layers, good throughput

What's optimal (78% of time)

GEMM at CPI=0.179, 82-84% retiring, 89% cycles with 3+ ports utilized. Near-theoretical peak. 256-bit INT ops = 23% of retired instructions.

What's left to optimize (22% of time)

im2col + memcpy: 15% — front-end bound from branch-heavy memcpy dispatch for small rows. Every fusion attempt we tried was slower (sequential bulk ops win).
Conv1 FP32: 4% — could be moved to INT8 path by quantizing the FP32 input.

8. Cache Behavior (perf stat, per inference)

Metric	Value	Interpretation
L1 loads	8.9M	—
L1 miss rate	**0.97%**	99% served from L1 (48 KB)
L2 demand reads	86.6K	Only 1% of L1 loads reach L2
L2 miss rate	**1.80%**	Almost nothing escapes L2 (1.25 MB)
LLC misses / inference	1,059	~68 KB DRAM traffic per inference
Overall Memory Bound	5.4%	Not a bottleneck at CIFAR-10 scale
DRAM BW utilized	2.0 / 37 GB/s	5.4% of max — compute bound, not BW bound

Why so cache-friendly: CIFAR-10 activations are tiny (8-64 KB). Even the largest im2col buffer (576 KB for layer1) fits in L2. 16-OC tiling keeps the active weight slice (9-72 KB) in L1. INT8 = 4× less data vs FP32.

9. Final Numbers

Method	1-Thread	6-Thread	Accuracy	Model Size
PyTorch FP32	11.18 ms	6.50 ms	95.54%	44.69 MB
ONNX FP32	9.75 ms	2.49 ms	95.54%	44.69 MB
ONNX INT8 Dynamic	4.26 ms	2.74 ms	95.48%	11.23 MB
ONNX INT8 Static	2.56 ms	1.08 ms	95.43%	11.23 MB
C++ Ternary Engine	**4.48 ms**	**2.11 ms**	**94.46%**	**3.56 MB**

38×
vs naive C++ baseline

1.3×
faster than ONNX INT8 Dynamic (6T)

3.2×
smaller than ONNX INT8

Generated from VTune uarch-exploration, perf stat, and debug walkthrough data. All numbers measured on Intel i5-13400F, Linux 6.8, GCC -O3 -march=native.